# IERG4320/IEMS5726 — Chapter 3: Data Representation I — Cheat Sheet

Prof. YAN Wenjing, Dr. Danny Yip | Department of Information Engineering, CUHK

## Data Preprocessing (数据预处理)

在使用数据之前先把数据整理干净，让模型能更好地学习。

### Why? (为什么要预处理？)

- Accuracy (准确性：数据对不对？)
- Completeness (完整性：有没有缺失？)
- Consistency (一致性：各处数据是否矛盾？)
- Timeliness (时效性：数据是否过时？)
- Believability (可信度：数据可靠吗？)
- Interpretability (可理解性：能不能看懂？)

### 4 Goals (四大目标)

1. **Data Cleaning** (清洗): fix/remove wrong, duplicate, incomplete data
2. **Data Integration** (整合): merge data from multiple sources
3. **Data Reduction** (降维): reduce data size while keeping info
4. **Data Transformation** (转换): change format/structure/-values

### Data Wrangling Steps (数据整理六步)

1. Discovering (了解数据长什么样)
2. Structuring (组织原始数据)
3. Cleaning (统一格式、去掉异常值)
4. Enriching (能不能补充更多数据？)
5. Validating (检查质量和一致性)
6. Publishing (准备好给下游使用)

### Preprocessing Pipeline (预处理流程)

加载数据 → 数据整理 → 处理缺失值 → 处理分类变量 → 去重 → 划分训练/验证/测试集 → 归一化 → (可选)特征选择

## Train/Val/Test Split (数据集划分)

- **Training set** (训练集): train the model (用来训练模型)
- **Validation set** (验证集): tune hyperparams (调参、选模型)
- **Testing set** (测试集): final evaluation (最终评估，只用一次)

```
from sklearn.model_selection import train_test_split
# First split: 80% train+val, 20% test
X_1, X_test, y_1, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
# Second split: 80% train, 20% val
X_train, X_val, y_train, y_val = train_test_split(
    X_1, y_1, test_size=0.2, random_state=0)
```

## Cross Validation (交叉验证)

把数据分成k份，轮流用其中1份做验证，其余做训练，重复k次取平均。

目的：更可靠地评估模型效果，避免"运气好"。

### Model selection example (选模型技巧):

选CV和Test表现都不错且差距小的模型（如M3/M4），不选差距太大的（如M1/M2）。

## Data Imputation (缺失值填补)

数据缺失很常见（漏填、拒绝回答、设备故障等），不能直接扔掉，要想办法填上。

### Types of Missingness (缺失类型)

- **MCAR** (完全随机缺失): missing by pure chance
  像随机在数据上挖洞，跟数据本身无关
- **MAR** (随机缺失): depends on other observed variables
  缺失跟其他已知变量有关，如性别影响是否回答某问题
- **MNAR** (非随机缺失): depends on the missing value itself
  缺失跟缺的值本身有关，如作弊的人不回答"你作弊过吗"

### Method 1: Deletion (删除法)

```
# Drop rows by index
df.drop([0, 1])
# Drop columns by name
df.drop(columns=['A','B'], axis=1)
# Drop any row with NaN
df.dropna()
```

简单粗暴，但会丢失信息。

### Method 2: Mean/Median/Mode (均值/中位数/众数填补)

```
import numpy as np
from scipy import stats
# Fill with mean
df.replace({'D': np.nan}, np.mean(df['D']))
# Fill with median
df.replace({'B': np.nan},
    np.nanmedian(df['B']))
# Fill with mode
df.replace({'D': np.nan},
    stats.mode(df['D'])[0][0])
```

### Method 3: Regression (回归填补)

用其他列建回归模型来预测缺失值。

```
from sklearn import linear_model
df_train = df.dropna()
X = df_train.drop(columns=['D'])
regr = linear_model.LinearRegression()
regr.fit(X, df_train['D'])
regr.predict(df.drop(columns=['D']))
```

注意：可能引入变量间额外依赖关系。

### Method 4: Hot Deck (热卡填补)

用同一列中最近的有效值来填充（向前填充）。

```
# Forward fill: use last valid value
df.ffill(inplace=True)
```

### Method 5: Multiple Imputation (多重填补)

最精确的方法：建多个模型，综合分析不确定性。

```
from sklearn.experimental import (
    enable_iterative_imputer)
from sklearn.impute import IterativeImputer
imp = IterativeImputer(
    max_iter=10, random_state=0)
imp.fit([[1,2],[3,6],[4,8],
         [np.nan,3],[7,np.nan]])
X_test = [[np.nan,2],[6,np.nan],[np.nan,6]]
print(np.round(imp.transform(X_test)))
# [[ 1.  2.] [ 6. 12.] [ 3.  6.]]
```

模型学到第二列是第一列的2倍，自动补全。

### sklearn Imputers (sklearn填补工具)

- `SimpleImputer` (均值/中位数/众数/固定值)
- `IterativeImputer` (迭代回归，多变量)
- `KNNImputer` (用K近邻来填补)
- `MissingIndicator` (标记哪里缺失)

### Guidelines (Harrell 2001) (实用指南)

| Missing % | Strategy (怎么处理) |
|---|---|
| $\leq 5\%$ | Median/Mode (简单填就行) |
| 5–15% | Multiple imputation if correlated (看情况) |
| 15–30% | Multiple imputation (必须用多重填补) |
| $> 30\%$ | Consider dropping the variable (缺太多，考虑删) |

## Categorical Variables (分类变量处理)

机器学习需要数字输入，所以要把文字类别转成数字。

### Label Encoding (标签编码)

把每个类别映射成一个数字。如 male→1, female→0

```
from sklearn import preprocessing
le = preprocessing.LabelEncoder()
df['A'] = le.fit_transform(df['A'])
df['A'].unique()
```

### Binning (分箱/分桶)

把连续值分成几个区间。如 age: 0–17→0, 18–64→1, 65+→2

```
bins = [0, 17, 64, np.inf]
names = [0, 1, 2]
```

```
df['ageRange'] = pd.cut(
    df['age'], bins, labels=names)
```

### Ordinal Encoding (有序编码)

类别有大小关系时使用。如 red→1, green→2, blue→3

```
df.name = pd.Categorical(df['name'])
df['code'] = df.name.cat.codes
```

### One-Hot Encoding (独热编码)

每个类别变成一列0/1。red→{1,0,0}, green→{0,1,0}, blue→{0,0,1}

**Dummy encoding** (哑变量): drop one column to avoid redundancy.

red→{1,0}, green→{0,1}, blue→{0,0}（少一列，减少冗余）

### Combine Levels (合并类别)

频率很低的类别可以合并到一起，减少稀疏性。

## Duplication Removal (去重)

### Drop Duplicate Rows (删重复行)

```
data = df.drop_duplicates()
```

### Filter Constant Columns (删常数列)

如果一列所有值都一样，没有信息量，删掉。

```
data = df.loc[:,
    (df != df.iloc[0]).any()]
```

### Filter Correlated Columns (删高相关列)

两列高度相关（如温度和温度×2），留一个就够了。

```
def correlation(dataset, threshold):
    col_corr = set()
    corr_matrix = dataset.corr()
    for i in range(len(corr_matrix.columns)):
        for j in range(i):
            if abs(corr_matrix.iloc[i,j]) \
                > threshold:
                colname = \
                    corr_matrix.columns[i]
                col_corr.add(colname)
    return col_corr
```

## Data Normalization (数据归一化)

不同特征量纲差距大（如年龄1–100 vs 收入10000–1000000），需要统一尺度。

### 1. Min-Max (最小最大归一化)

缩放到[0, 1]区间：$x' = \frac{x - \min}{\max - \min}$

```
from sklearn import preprocessing
scaler = preprocessing.MinMaxScaler()
X_scaled = scaler.fit_transform(X_train)
```

### 2. Mean Normalization (均值归一化)

$x' = \frac{x - \text{mean}}{\max - \min}$，结果在[-1, 1]附近

```
df_means = df.mean(axis=0)
min_max_range = df.max(0) - df.min(0)
df_scaled = (df - df_means) / min_max_range
```

### 3. Z-score / Standardization (标准化)

$x' = \frac{x - \mu}{\sigma}$，均值变0，标准差变1。最常用！

```
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
df_scaled = ss.fit_transform(df)
```

### 4. Unit Length (单位长度缩放)

让每个特征向量的长度（L2范数）变成1

```
df_scaled = df / np.sqrt(
    np.square(df).sum(axis=0))
```

## Feature Selection (特征选择)

数据太多列怎么办？挑出最有用的列，扔掉没用的，让模型更快更准。

### Why? (为什么要特征选择？)

- Remove irrelevant features (去掉没用的列)
- Improve accuracy (提高准确率)
- Reduce cost (省计算资源)
- Simplify model (模型更简单易懂)

### Two Approaches (两大类方法)

1. **Select columns/rows** (直接选列/行)
   不改变原始特征，只挑子集
2. **Dimensionality reduction** (降维)
   把多个特征压缩成少数几个新特征（PCA/LDA）

### Search Directions (搜索方向)

- **SFG** (前向): start empty, add features one by one
- **SBG** (后向): start full, remove features one by one
- **BG** (双向): add and remove simultaneously
- **RG** (随机): random subsets

### Selection Criteria (评价标准)

- Information gain (信息增益)
- Distance measure (距离度量)
- Correlation (Pearson) (相关系数)
- Consistency (一致性)
- Accuracy (准确率)

### Wrapper Method (包裹法)

用分类器本身来评估哪些特征组合最好。

```
from mlxtend.feature_selection import (
    SequentialFeatureSelector as SFS)
sfs = SFS(RandomForestClassifier(n_jobs=8),
    k_features=5,    # pick 5 features
    forward=True,    # forward search
    floating=False,
    scoring='roc_auc', cv=3)
sfs = sfs.fit(X_train.values, y_train)
selected = X_train.columns[
    list(sfs.k_feature_idx_)]
```

### Embedded: LASSO (嵌入法：L1正则化)

L1正则化会自动把不重要的特征权重压成0。

```
from sklearn.linear_model import (
    LogisticRegression)
from sklearn.feature_selection import (
    SelectFromModel)
from sklearn.preprocessing import (
    StandardScaler)
scaler = StandardScaler()
scaler.fit(X_train.values)
sel = SelectFromModel(
    LogisticRegression(C=1, penalty='l1'))
sel.fit(scaler.transform(X_train.values),
    y_train)
selected = X_train.columns[sel.get_support()]
X_train = sel.transform(X_train)
X_test = sel.transform(X_test)
```

## PCA (主成分分析)

把很多特征压缩成少数几个"主成分"，保留尽可能多的信息（方差）。

> 核心思想：找到数据方差最大的方向，投影上去。
> PCA是无监督的（不看标签），返回最多min(行数,列数)个主成分。

- PC basis = eigenvectors of covariance matrix (主成分=协方差矩阵的特征向量)
- Larger eigenvalue = more important PC (特征值越大越重要)
- Pick top PCs covering $\geq 85\%$ variance (选前几个，覆盖85%+方差)

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn import datasets
digits = datasets.load_digits()
# Standardize first!
X = StandardScaler().fit_transform(
    digits.data)
# Keep 85% of variance
pca = PCA(n_components=0.85, whiten=True)
X_pca = pca.fit_transform(X)
```

## LDA (线性判别分析)

和PCA类似做降维，但LDA是有监督的——会利用类别标签，让不同类尽量分开。

> **PCA vs LDA**:
> PCA：无监督，最大化方差（保留信息最多）
> LDA：有监督，最大化类间距离（分类效果最好）
> PCA输出最多min(n,p)个PC；LDA输出最多K−1个LD（K=类别数）

```
from sklearn.discriminant_analysis import (
```

```python
    LinearDiscriminantAnalysis as LDA)
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import (
    train_test_split)
# Split and scale
X_train, X_test, y_train, y_test = \
    train_test_split(X, y,
    test_size=0.2, random_state=0)
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
# LDA: reduce to 1 dimension
lda = LDA(n_components=1)
X_train = lda.fit_transform(X_train, y_train)
X_test = lda.transform(X_test)
```

注意：LDA的fit需要传y（标签），PCA不需要。

## Other Methods (其他降维方法)

- **t-SNE** (保留局部结构，适合可视化)
- **UMAP** (比t-SNE更快，兼顾全局)
- **SVD** (奇异值分解，PCA的推广)

## Quick Reference (速查总结)

| **Task** (任务) | **Key Code** (关键代码) |
|---|---|
| Split data (划分) | `train_test_split()` |
| Fill NaN (填缺失) | `SimpleImputer / IterativeImputer` |
| Encode cat. (编码) | `LabelEncoder / pd.cut / cat.codes` |
| Deduplicate (去重) | `drop_duplicates()` |
| Normalize (归一化) | `MinMaxScaler / StandardScaler` |
| PCA (主成分) | `PCA(n_components=0.85)` |
| LDA (判别) | `LDA(n_components=1)` |
| Feature sel. (特征选择) | `SFS / SelectFromModel(LASSO)` |

**Preprocessing Pipeline** (完整流程):
Load → Wrangle → Impute → Encode → Deduplicate → Split → Normalize → Feature Select
加载 → 整理 → 填缺失 → 编码 → 去重 → 划分 → 归一化 → 选特征